Product spec & implementation plan — AI Profile Autofill (MVP) + Job Search & Matching

Role: senior engineer + advisor — honest, practical, no fluff.

This document is the updated, copy-ready spec that adds **Job Search, Filtering, and Matching** to the previously agreed AI Autofill & Smart Reply Chrome extension MVP. It covers feature additions, technical approach, API design, UI changes, backend responsibilities, legal concerns, and updated checklist & timeline.

One-line summary

A Chrome extension that uses AI to **generate and autofill profile and form content** (bios, skills, gig descriptions), provide **smart replies**, and now **discover jobs, filter, and match** users to relevant web2 and web3 job listings — with a local-first UX, a minimal backend proxy for API keys, and a freemium model (Hugging Face free tier for basic users; OpenAI for premium).

New & Updated MVP feature list (final)

1. **AI Autofill Engine** — detect fields, generate, insert.
2. **Tone Selector** — choose Professional / Casual / Persuasive / Friendly.
3. **Reusable Snippets** — save/manage local snippets.
4. **Multi-profile Support** — Freelancer / Job Seeker / Founder personas.
5. **Quick Rewriter** — highlight text → rewrite/shorten/expand.
6. **Keyword Optimizer** — suggest SEO keywords for gigs/jobs.
7. **Consistency Mode (lite)** — reuse tone/keywords across fills.
8. **Popup Disclaimer** — warns about anti-automation risk; proceed/skip & “don’t show again.”
9. **Copy–Paste Fallback** — generate in popup for manual paste.
10. **Smart Reply Assistant** — generate suggested replies for messages/chats (inline or popup).
11. **Auto-detect Page Context** — detect platform type and tweak prompts.
12. **Job Search & Aggregation** — search jobs from APIs and curated RSS feeds.
13. **Job Scraping (optional, v2)** — controlled scraping for sources without APIs (use cautiously).
14. **Filters & Saved Searches** — keyword filters, location, remote, salary, web2/web3, tags.
15. **Job Matching & Scoring** — rank jobs by relevance using keyword and embedding-based matching.
16. **Quick Apply / Auto-fill Integration** — jump from job card to autofill-generated application content.
17. **Alerts & Saved Searches** — email/extension notifications for new matches (v2).

Architecture (textual) — updated

Client (Chrome extension, Vite + React + TS + Tailwind)
- Content script: scans page, detects fields, inserts generated text (or simulates typing).
- Popup UI: tabs for Home (Autofill), Snippets, Replies, Jobs. Jobs tab shows search, filters, results, and quick-apply buttons.
- Local storage: `chrome.storage.local` for user prefs, snippets, saved searches, and per-domain settings.

Backend (minimal, Express/TS on Render)
- API proxy endpoints: `/generate`, `/jobs/search`, `/jobs/refresh`, `/jobs/source-status`, `/auth` (future), `/usage`, `/health`.
- Job Aggregator service (within backend): consumes APIs & RSS feeds, normalizes job data, caches results, deduplicates, and exposes search endpoints.
- Stores API keys for job APIs, Hugging Face/OpenAI keys, and handles rate-limits and caching.

- Optional: small Redis or in-process cache for job results (Render free-tier may limit options — start with in-memory cache, move Redis later).

Third-party
- Hugging Face (free tier) for AI generation.
- Job APIs & feeds (Indeed, Remotive, RemoteOK, WeWorkRemotely, Web3.career, CryptoJobsList, etc.).
- Optional proxies / rotating IPs for scraping (v2 paid).

Job Search & Matching — Technical approach (one after the other)

12) Job Search & Aggregation (MVP approach)
Goal: surface relevant job listings (web2 & web3) in the extension Jobs tab with filters and quick-apply capability.

Data sources (MVP):
- Prioritize free APIs and RSS feeds: Remotive, RemoteOK, WeWorkRemotely, Indeed (if available), CryptoJobsList, Web3.career, Remote3, etc.
- For sites without API, use RSS/JSON feeds if available. Avoid scraping in MVP to minimize legal risk.

Backend flow:
- Aggregator fetches from each source on schedule (e.g., every 15–60 minutes).
- Normalize job structure: `{ id, title, company, location, remote, url, postedAt, description, tags, source, salary }`.
- Cache results and deduplicate using hashing of title+company+url.
- Expose `POST /jobs/search` that accepts filters & returns paginated results.

Frontend flow:
- Jobs tab: search input, filter controls (Remote toggle, Location, Min Salary, Web3 tag, Keywords), result list with paging.
- Quick Apply button fetched with job details → triggers autofill pipeline (generate tailored answers, insert or show popup).

Endpoint example:
`POST /jobs/search`
Request body:
```json
{ "q":"react developer", "filters":{ "remote":true, "web3":true, "location":"", "minSalary":0 }, "page":1, "perPage"
```
```
Response:
```json
{ "ok":true, "results":[{ "id":"...", "title":"...", "company":"...", "snippet":"...", "url":"..." }], "total":123 }
```

Caching & rate limiting: cache per-source and globally; store last-fetch timestamps to avoid hitting APIs too often.

13) Job Scraping (optional, v2)
Goal: cover sources that don't provide APIs or feeds.

Approach & risks:
- Use scraping only for smaller sources or with explicit permission. Respect robots.txt and site ToS.
- Use headless browsers (Puppeteer) only on backend aggregator, behind proxy pool (paid).
- Scraping adds complexity: rotate IPs, handle site changes, and maintain selectors. Prefer APIs first.

Recommendation: delay scraping until there’s clear demand & budget for proxies and maintenance.

14) Filters & Saved Searches
Goal: make job discovery productive & repeatable.

Core filters (MVP):
```

- Keywords (title, description)
- Remote / On-site / Hybrid
- Location (city, country)
- Web2 / Web3 toggle (tag)
- Salary range (if available)
- Company size / contract type (if available)

Saved searches & alerts: allow users to save search queries locally; optionally schedule background checks and send notifications (v2).

15) Job Matching & Scoring
Goal: score each job by relevance to the user profile and preferences.

Approach (MVP -> improved):

MVP (keyword + heuristic matching):
- Score = weighted sum of matches:
- Keyword matches in title (strong) and description (medium).
- Profile tag matches (skills from user profile).
- Recency bonus (newer posts higher score).
- Simple TF-IDF or token overlap is enough for MVP.

Improved (v2, embedding-based):
- Generate embeddings for user profile (skills, bio, keywords) and job description using Hugging Face or OpenAI embeddings.
- Compute cosine similarity between profile embedding and job embedding; use as relevance score.
- Combine with heuristic signals (remote, salary) into final score.

Implementation notes: cache embeddings per job; compute embeddings on ingestion to save inference time on search.

16) Quick Apply / Auto-fill Integration for Jobs
Goal: let users apply faster by generating tailored application content from the job posting and inserting it into the job's application fields.

Flow:
- When user clicks **Quick Apply** on a job card: fetch job details (title, description, required skills).
- Build prompt for AI:

> "Write a 150-word cover message for a {profile} with skills {skills} applying to {jobTitle} at {company}. Make it {tone} and reference {2-3 keywords}."
- Generate content via backend `/generate` and insert via autofill or show popup for copy/paste.
- Offer "Save as Snippet" and "Send Reply" options.

Edge cases: some job forms block programmatic input → fallback to copy/paste UI.

17) Alerts & Saved Searches (v2)
Goal: notify users of new jobs matching saved searches.

Approach:
- Backend job aggregator triggers change detection; when new items match saved queries, store alerts.
- For MVP (no email infra), show notifications inside extension or use browser notifications.
- For email alerts or mobile push, add third-party service and authentication in v2.

Backend API design (updated)

Base: `https://api.yourdomain.com`

New / Updated Endpoints

- `POST /jobs/search` — search aggregated jobs.
- `GET /jobs/:id` — get full job details & raw source link.
- `POST /jobs/refresh` — manual trigger to refresh aggregator for a source (admin use).
- `GET /jobs/sources` — status of configured sources and last fetch time.

Existing endpoints retained: `/generate`, `/health`, `/usage`, `/auth`

Server responsibilities (updated):
- Job ingestion scheduler & normalizer.
- Embedding computation (v2) & caching.
- Rate-limit per-source and per-user.
- Deduplication & basic enrichment (extract key skills via AI).

UI changes — Jobs tab & flows

Jobs tab (popup / panel)
- Search bar + Filters row (keywords, remote toggle, Web2/Web3 toggle, location, salary).
- Result list: job card shows title, company, remote/location, short snippet, tags, postedAt, and Quick Apply button.
- Quick Apply opens a small modal: generated cover letter + "Insert into form" / "Copy" buttons.
- Saved Searches panel & Alerts badge (v2).

Small-screen constraint: keep job results minimal — show 4–6 cards per popup page; optionally open a side panel or new tab for longer browsing.

Storage & security (job features)

- **Cache job results** server-side and minimize storage in extension.
- **Privacy:** saved searches and profile data are stored locally by default. If you enable server-side saved searches or alerts, clearly inform users and allow deletion.
- **Scraping legal risk:** document which sources you scrape, respect robots.txt, and provide opt-out for domains. Prefer APIs to avoid ToS issues.

Deployment & hosting (job features)

- **Backend:** Render is fine for MVP aggregator but note: frequent source polling and scraping can exceed free limits. Consider small VPS or server with scheduled jobs if needed.
- **Caching:** In-memory cache OK for tiny scale; add Redis/managed cache when scaling.
- **Rate-limits & proxies:** scraping requires rotating proxies (paid) — avoid for MVP.

Testing plan (jobs)

- Unit test normalizer & dedup logic.
- Integration tests for each job source (mock responses).
- E2E tests to validate Quick Apply → autofill workflow using dummy pages.
- Manual verification for web3 job sources (they’re inconsistent in schema).

MVP vs Phase 2 (priority) — updated

Phase 1 (MVP) — deliver first:
- AI Autofill Engine (core).
- Tone Selector.
- Popup Disclaimer + Copy–Paste fallback.
- Reusable Snippets (local).
- Smart Reply (popup mode).
- Backend proxy (Render) using Hugging Face free tier.
- **Job Search (MVP): aggregator of 3–5 free APIs/RSS sources, filters, job cards, Quick Apply (autofill integration).**

Phase 2 (after launch) — add:
- Inline Smart Replies.
- Keyword optimizer improvements.
- Consistency Mode advanced.
- Premium tier with OpenAI GPT-4o-mini, billing, user accounts.
- Embedding-based matching & alerts.
- Scraping with proxies (paid) for extra sources.
- Analytics + caching + Redis + dedicated job crawler infra.

Developer checklist & next steps (actionable) — updated

1. Repo bootstrap: Vite + React + TS + Tailwind. Create extension scaffold (manifest v3).
2. Implement content script: detect fields + simple UI message passing.
3. Build popup UI: tone selector, profile selector, snippet viewer, generate/insert buttons.
4. Set up minimal Express backend on Render with `POST /generate` that proxies to Hugging Face.
5. Wire extension to backend; test generation flow on a local dummy form site.
6. Implement popup disclaimer and per-domain storage.
7. Add quick rewriter (context menu) and smart reply popup.
8. **Implement job aggregator:** integrate 3 job APIs/RSS, create `/jobs/search` endpoint, frontend Jobs tab.
9. Add quick apply (autofill integration) for job cards.
10. QA across sample pages; polish UX.
11. Prepare privacy policy & install instructions.
12. Beta test with 10–20 users; collect feedback and iterate.

Updated timeline (very lean, solo)

- Day 0–3: Scaffolding & content script basics.
- Day 4–7: Popup UI + tone/profile + generate flow using mock backend.
- Day 8–11: Backend proxy + Hugging Face integration; connect end-to-end.
- Day 12–16: Snippets + disclaimer + copy/paste fallback + Smart Reply popup.
- Day 17–22: Job aggregator (3 sources) + Jobs tab frontend + Quick Apply wiring.
- Day 23–28: Testing, polish, small beta, and docs.

Legal & Privacy (jobs)

- Clearly indicate which sources you fetch and whether data is scraped.
- Respect robots.txt and API terms.

- Provide easy mechanisms for users to delete saved searches and snippets.
- Update Privacy Policy to reflect job ingestion & optional alerts.

Final notes — brutal honesty (jobs added)

- Adding job discovery makes the product more compelling and sticky, but it increases backend complexity and maintenance.
- Start small: 3 reliable sources (a mix of web2 + web3) and iterate. Avoid scraping until you have budget and clear demand.
- Embedding-based matching is a strong differentiator — add in v2 when you can afford embedding compute or caching.

If you want, I can:
- Generate `manifest.json` + minimal content script + popup React skeleton including a Jobs tab.
- Compile a short list of 10 job APIs/feeds you can plug into the aggregator next.